

Problem 1.**Airport data:**

- What is the α if the distribution is power law?

$\alpha = 1.612$, with $x_{\min} = 1$ route. Maximum-likelihood estimate $\hat{\alpha} = 1 + n / \sum_i \ln(x_i / x_{\min})$, $x_{\min} = \min_i x_i$, $n = 3409$.

- What is the λ if the distribution is exponential?

$\lambda = 0.0504$. MLE $\hat{\lambda} = 1/\bar{x} = 1/19.845$.

- What is $[a, b]$ if the distribution is uniform?

$[a, b] = [1, 915]$, the minimum and maximum route counts (MLE).

- What is μ and σ if the distribution is normal?

$\mu = 19.845$, $\sigma = 53.506$ (sample mean and sample standard deviation with $n - 1$).

- Screen shot of the visualization of distribution of data

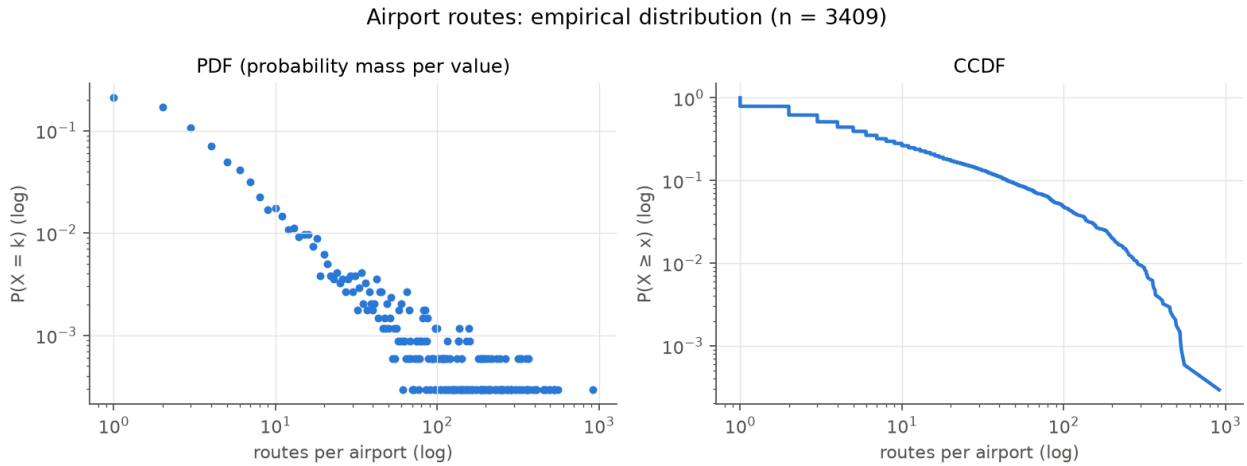


Figure 1: Airport routes. Left: share of airports with exactly k routes. Right: share with at least x routes (CCDF). Both are on log-log axes. The median airport has 4 routes and 38% have only 1 or 2, while the busiest has 915. The mean (19.8) is five times the median and the skewness is 6.0, so a few hubs hold most of the routes.

(These can be CDF, PDF, or QQplot)

For each model below I drew $n = 3409$ random values from the fitted distribution (seed 6330). Left panel: CCDF of the data and of the model sample on log-log axes. Right panel: QQ plot of model-sample quantiles against data quantiles; points on the dashed line $y = x$ mean the model reproduces the data. D is the two-sample Kolmogorov–Smirnov distance (largest gap between the two CDFs; 0 = identical).

- Screen shot of the visualization of distribution of data if it was power law

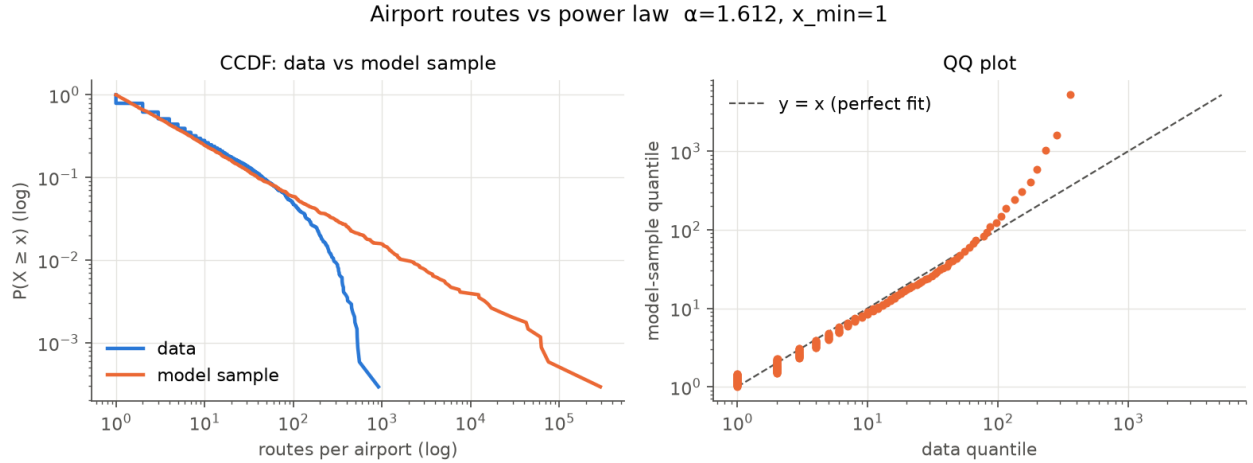


Figure 2: Power law ($\alpha = 1.612$, $x_{\min} = 1$). The CCDF follows the data from 1 to about 60 routes, and the QQ points lie on $y = x$ over that range. Above about 100 routes the model keeps going and produces airports with 10^4 to 10^5 routes, while the data bends down and stops at 915. $D = 0.209$.

- Screen shot of the visualization of distribution of data if it was exponential

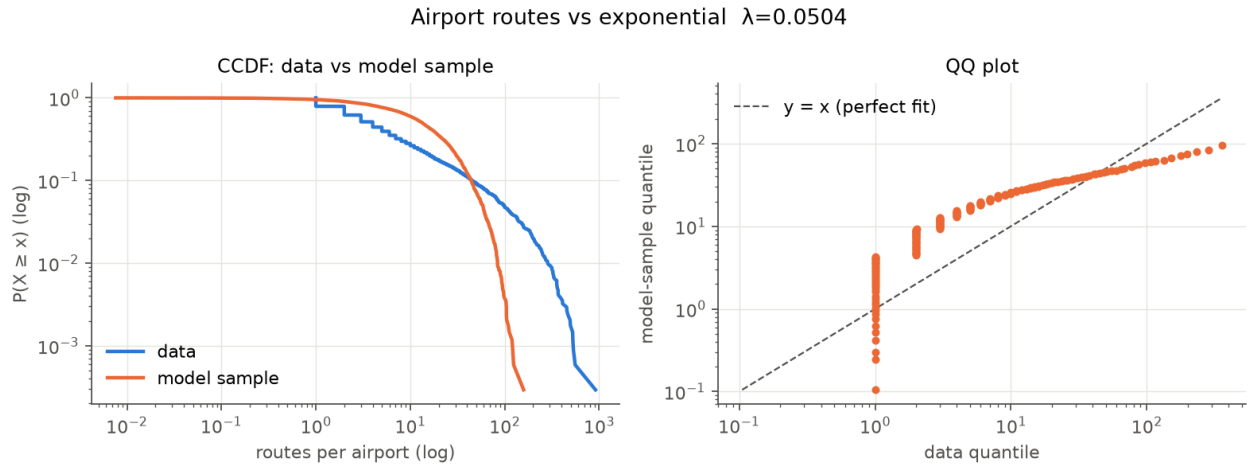


Figure 3: Exponential ($\lambda = 0.0504$). The model's sample goes below 1 route, which the data never does, and its tail dies out at about 150 routes. The QQ points are too flat: the model gives small airports too many routes and hubs too few. $D = 0.379$.

- Screen shot of the visualization of distribution of data if it was uniform

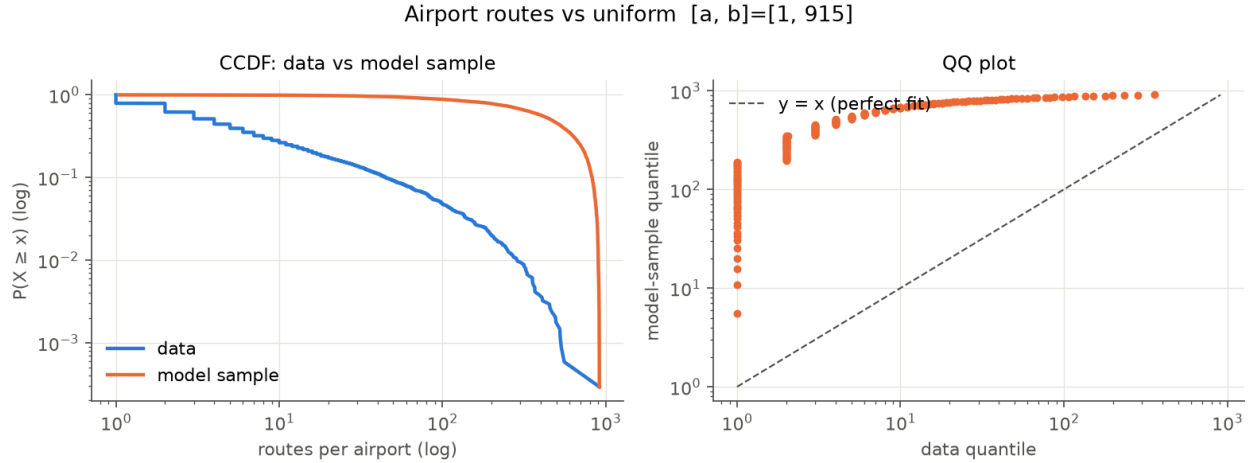


Figure 4: Uniform on $[1, 915]$. A typical sampled airport has about 460 routes, but 99% of real airports have fewer than 285. The model's CCDF stays near 1 until the right edge, and the QQ points lie far above $y = x$ for every quantile. $D = 0.857$, the worst of the four.

- Screen shot of the visualization of distribution of data if it was normal

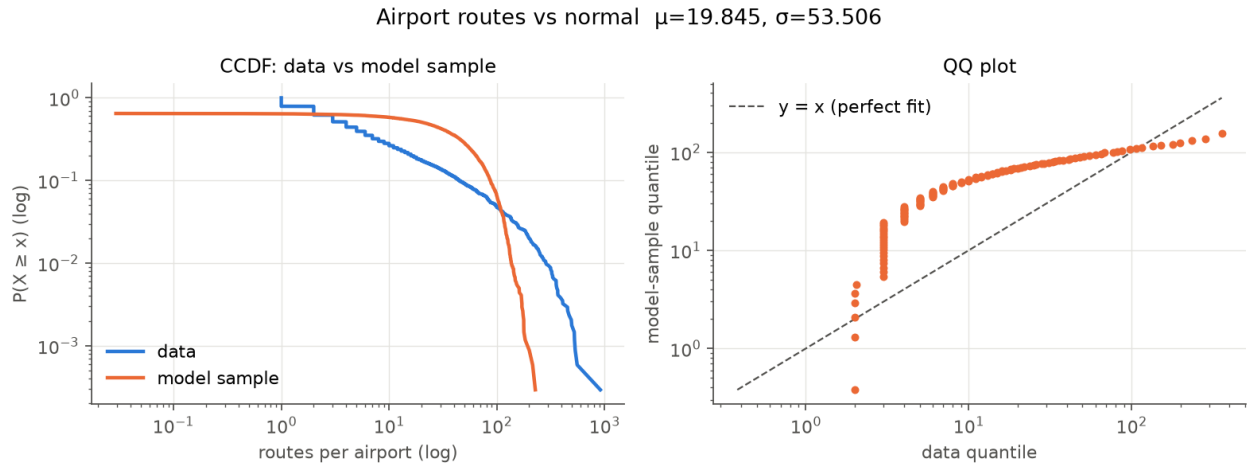


Figure 5: Normal ($\mu = 19.8, \sigma = 53.5$). With σ almost three times μ , 36% of the model's mass is below zero, which is impossible for a route count. The log axes can only show the positive part. The model's tail is also thin and ends near 200 routes. $D = 0.358$.

- What is the distribution of data based on your observation? Discuss

	power law	exponential	uniform	normal
KS distance D (data vs model sample)	0.209	0.379	0.857	0.358

The airport data follows a power law, with a cutoff in the far tail. It is the only one of the four models whose CCDF is a straight line on log-log axes. The data's CCDF is close to straight for more than two decades (1 to about 100 routes), and the power-law sample matches it there. The other three models fail for structural reasons, not because a parameter is slightly off:

- **Uniform** says every route count from 1 to 915 is equally likely, but most airports are tiny.

- **Normal** is symmetric and allows negative counts, while the data is bounded below at 1 and skewed to the right (skewness 6.0).
- **Exponential** has the right shape (many small, few large), but its tail falls off too fast. With $\lambda = 0.05$, an airport with 500 routes has probability about e^{-25} , yet the data has five airports with more than 500.

The power law is not a perfect fit. $\hat{\alpha} = 1.61$ is below 2, so the fitted model has an infinite mean and its samples contain airports with 10^5 routes. Real airports cannot grow that large: there are only 3409 airports to fly to, and runways, gates and slots limit the busiest hubs. That is why the data's CCDF bends down above about 100 routes. The best description is a power law with an upper cutoff, which is typical of networks that grow by preferential attachment: new routes go mostly to airports that already have many routes.

Movie Rating data:

- What is the α if the distribution is power law?
 $\alpha = 1.851$, with $x_{\min} = 1.9$ (the lowest average vote). Same MLE as for the airports, $n = 4392$.
- What is the λ if the distribution is exponential?
 $\lambda = 0.1606$. MLE $\hat{\lambda} = 1/\bar{x} = 1/6.227$.
- What is $[a, b]$ if the distribution is uniform?
 $[a, b] = [1.9, 8.5]$, the minimum and maximum votes (MLE).
- What is μ and σ if the distribution is normal?
 $\mu = 6.227$, $\sigma = 0.893$ (sample mean and sample standard deviation with $n - 1$).
- Screen shot of the visualization of distribution of data

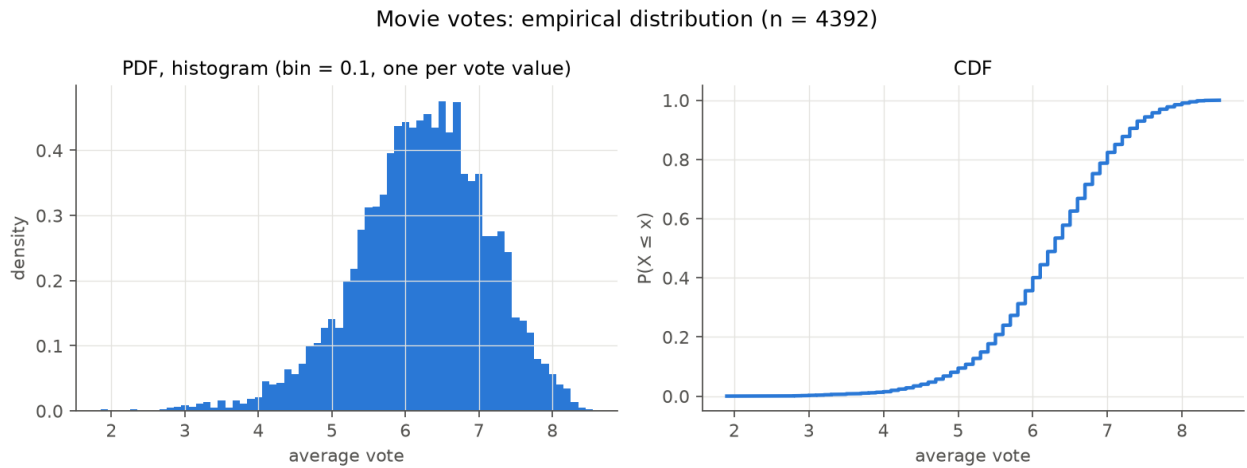


Figure 6: Movie votes. Left: histogram with one bin per possible vote (votes are rounded to 0.1). Right: empirical CDF. The distribution has one peak and is nearly symmetric around 6.3 (median 6.3, mean 6.23). The left tail is somewhat longer (skewness -0.48): a few movies are rated 2 to 4, and none are above 8.5.

(These can be CDF, PDF, or QQplot)

For each model below I drew $n = 4392$ random values from the fitted distribution. Left panel: PDF (histogram) of the data and of the model sample. Right panel: QQ plot as for the airports. D is the KS distance.

- Screen shot of the visualization of distribution of data if it was power law

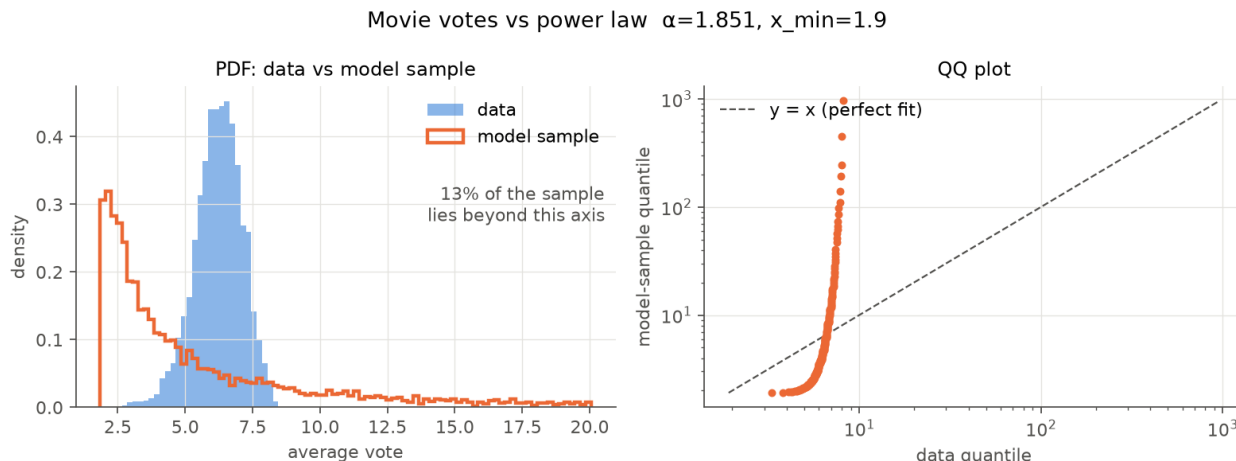


Figure 7: Power law ($\alpha = 1.851$, $x_{\min} = 1.9$). The model puts its highest density at the lowest vote, which is where the data has almost nothing, and 13% of its sample lies above 20, which is not a possible rating. The QQ plot is on log axes because the sample goes up to about 1000. $D = 0.485$.

- Screen shot of the visualization of distribution of data if it was exponential

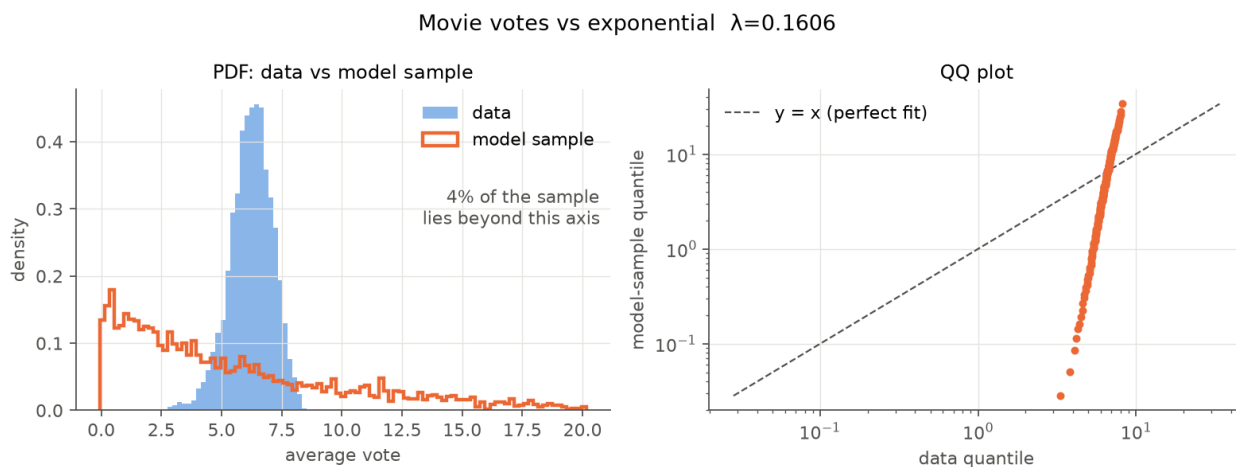


Figure 8: Exponential ($\lambda = 0.1606$). The model also peaks at zero and has a long right tail (4% of its sample is above 20). In the QQ plot the points cross $y = x$ at a steep angle, so the model's spread is far larger than the data's. $D = 0.490$.

- Screen shot of the visualization of distribution of data if it was uniform

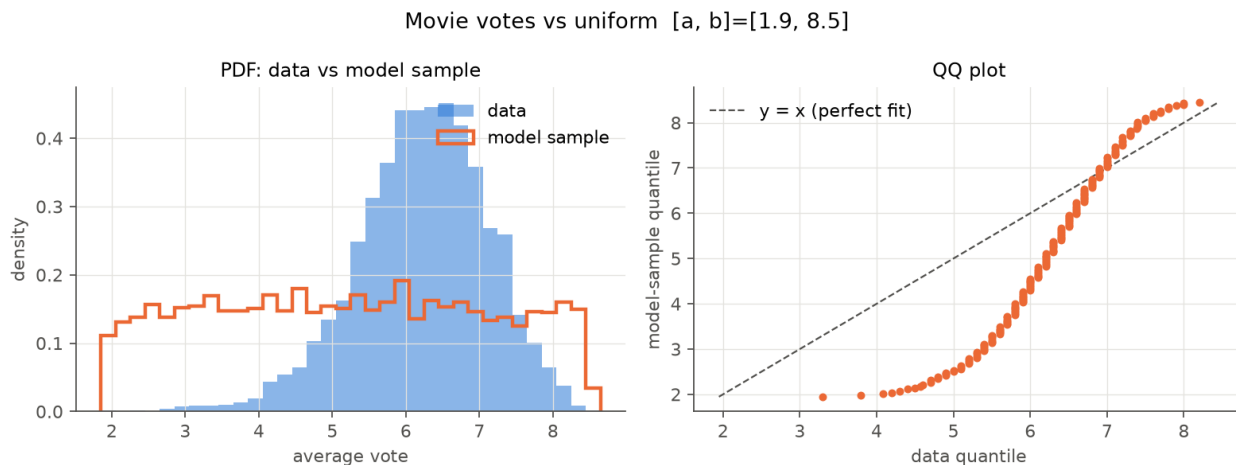


Figure 9: Uniform on $[1.9, 8.5]$. The model is flat, but the data is concentrated between 5 and 7.5. The QQ plot is an S-shape: the model's low quantiles are too low and its high quantiles too high. $D = 0.394$.

- Screen shot of the visualization of distribution of data if it was normal

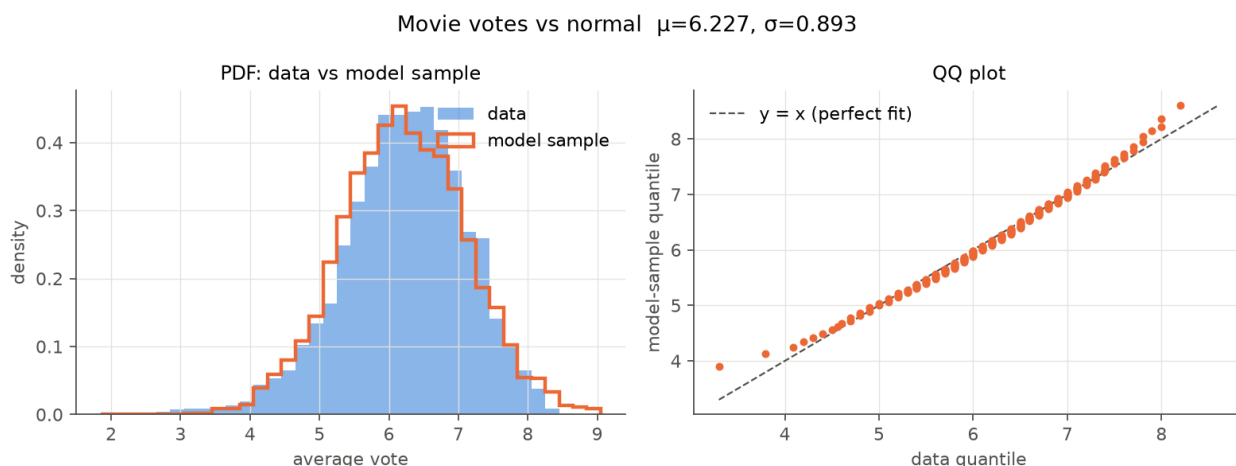


Figure 10: Normal ($\mu = 6.227, \sigma = 0.893$). The sample traces the same bell as the data, and the QQ points lie on $y = x$ from about 4.5 to 7.5. The only departures are at the ends: the data's lowest votes are lower than the model predicts (longer left tail), and the data stops at 8.5 while the model continues. $D = 0.055$, by far the smallest.

- What is the distribution of data based on your observation? Discuss

	power law	exponential	uniform	normal
KS distance D (data vs model sample)	0.485	0.490	0.394	0.055

The movie votes follow a normal distribution, approximately. The data has one peak near the middle of its range and is close to symmetric. The normal model predicts 68.3% of movies within $\pm 1\sigma$ and 95.4% within $\pm 2\sigma$; the data has 70.1% and 95.6%. In the QQ plot the points lie on the identity line through the bulk of the data, and the KS distance is about one seventh of the next-best model's.

The other three models fail for structural reasons. The power law and the exponential are both monotonically decreasing: their most likely value is the smallest one, so they predict mostly terrible movies and a long tail of ratings above 10. The uniform model ignores the peak altogether.

The fit is good but not exact, for two reasons. First, the data is bounded to $[1, 10]$ and slightly left-skewed: bad movies can drop further below the average than good movies can rise above it. Second, votes are rounded to one decimal, which gives the QQ plot its small steps. A normal shape is expected because each value is an *average* of many individual votes. By the central limit theorem an average of many independent ratings tends toward a normal distribution, whatever the distribution of each single rating.

Problem 2.

- Screenshot of distribution/histogram of the age

How the ages were computed. The birth dates have the form `dd-Mon-yy` (a few use a 4-digit year). Every 2-digit year is read as `19yy`: IMDB-WIKI was collected in 2015, so a year such as `20–24` cannot mean 2020–2024, and the counts per year decrease smoothly from the 1920s down to `00`, with no break that would mark a switch to the 2000s. The 129 rows with year `0000` (a missing-value placeholder) were dropped. Age is the number of completed years on 2015-01-01, the collection year. Keeping ages 1 to 99 leaves **458,015** people (mean 45.3, median 43, range 15 to 99). The minimum of 15 is a result of the `19yy` rule, since the latest possible birth year is 1999.

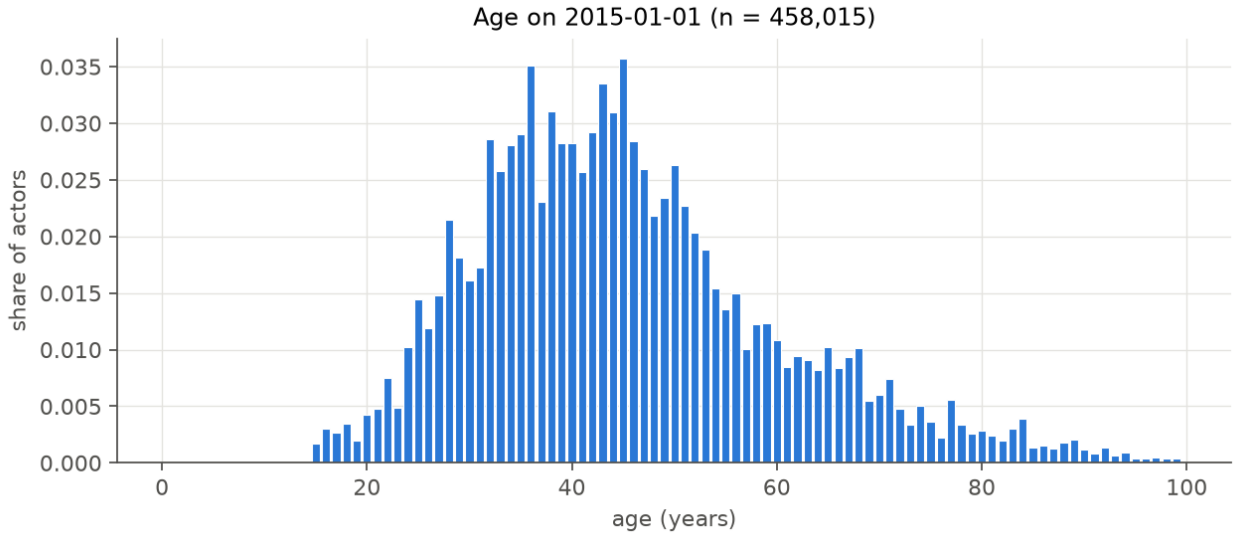


Figure 11: Age in 2015. One broad peak in the 30s and 40s and a long right tail up to 99; most of the mass is between 25 and 60.

- Screenshot of distribution/histogram of the first digit of age

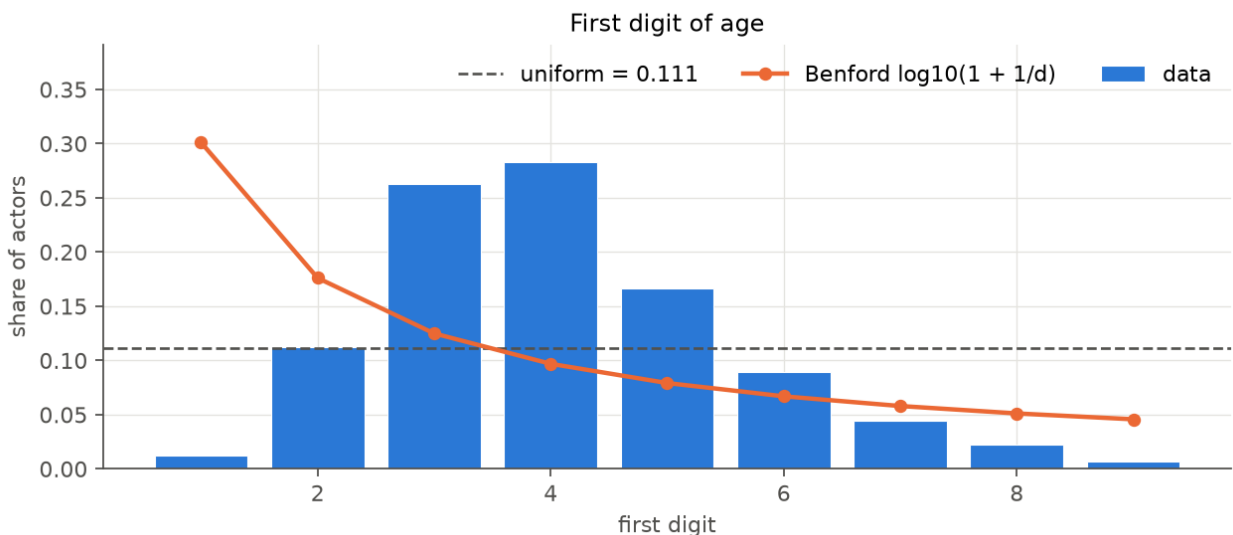


Figure 12: First digit of age, with the uniform level $1/9$ (dashed) and Benford's law (orange). The shares are 1.3%, 11.3%, 26.2%, 28.3%, 16.7%, 9.0%, 4.4%, 2.2% and 0.7% for digits 1 to 9. The data is far from uniform and also far from Benford: it peaks at 3 and 4, where Benford predicts a steady decline from 1.

- Screenshot of distribution/histogram of the last digit of age

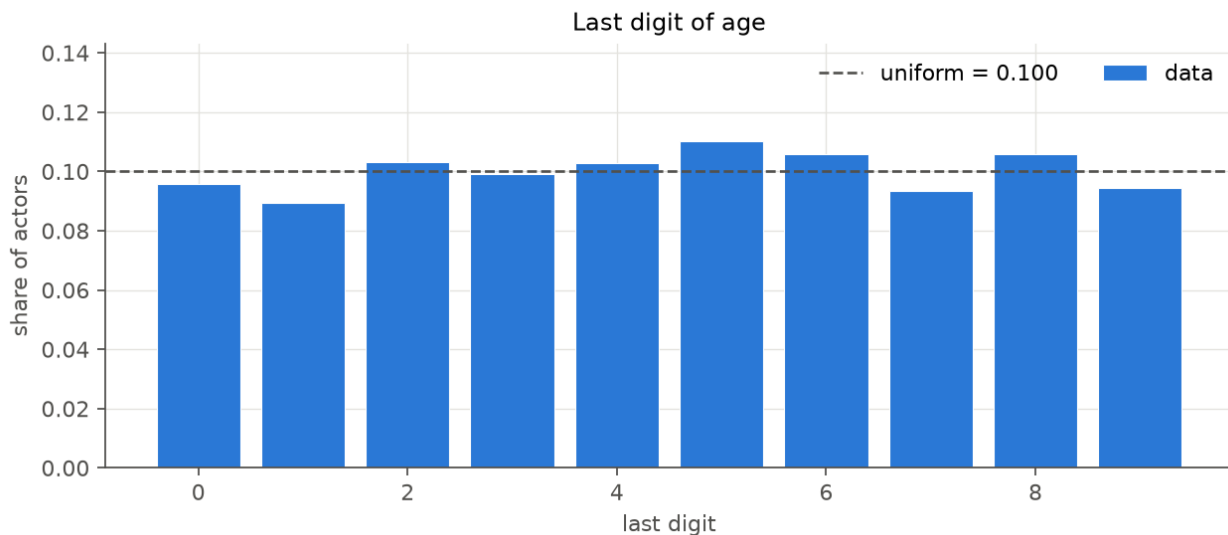


Figure 13: Last digit of age, with the uniform level $1/10$ (dashed). Every digit has a share between 8.95% and 11.01%, within 1.1 percentage points of 10%.

- Discussion (what are those distribution?) Why do you think it is like this?

The last digit is uniform (approximately); the first digit is not.

Last digit. This is expected. The last digit of an age is its value modulo 10. Every block of ten ages (20–29, 30–39, ...) contains each last digit exactly once. The age distribution changes slowly compared with a step of one year, so within any block of ten the ten ages have roughly equal counts, and summing over the blocks gives each last digit about 10%. The remaining deviations of up to ± 1.1 points come from year-to-year noise in how many actors were born in a given year (the jagged bars in the age histogram), not from any preference for particular digits.

First digit. This is also expected once you notice that, for ages 10 to 99, the first digit *is* the decade of life. Its histogram is therefore the age distribution grouped by decade, and it has the same shape as the age distribution: one peak (in the 30s and 40s) with a right tail. Digit 1 is almost empty because only ages 15 to 19 are possible and few actors are that young, and 9 is almost empty because few actors reach their 90s. For the first digit to be uniform, every decade from teens to 90s would need the same number of actors, which working actors clearly are not.

It is not Benford's law either. Benford's law (first digit d with probability $\log_{10}(1 + 1/d)$, so 1 is the most common at 30.1%) applies to data that spans several orders of magnitude, such as populations, prices or the airport route counts in Problem 1, where the log of the value is spread out roughly evenly. Age spans less than one order of magnitude (15 to 99) and is concentrated around 45, so its first digit shows where that one hump sits, not Benford's pattern. In short, the last digit is uniform because it throws away where the value is and keeps only its position within a block of ten. The first digit keeps exactly that information, so its distribution follows the shape of the age distribution.

Tooling. Python 3 / pandas / numpy / scipy / matplotlib; all estimates computed programmatically, none typed by hand; analysis code, figure code and write-up drafted with Claude Code. Code: `hw2/code/p1.py` and `hw2/code/p2.py`, listed below.

Code

Problem 1: hw2/code/p1.py

```
"""
Problem 1: fit power law, exponential, uniform and normal models to
  A. airport routes (routes per airport)
  B. movie votes (average TMDb rating)
and compare each fitted model against the data by drawing random samples from it.

Run from the repo root: uv run python hw2/code/p1.py
Writes figures to hw2/figures/ and prints every estimate used in the report.
"""

from pathlib import Path

import matplotlib

matplotlib.use("Agg") # headless: write PNGs only, no GUI backend needed
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import ks_2samp

HW_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = HW_DIR / "00_doc"
FIG_DIR = HW_DIR / "figures"
SEED = 6330
VOTE_VIEW_MAX = 20.0 # x-limit for movie PDFs; ratings live in [1, 10]

DATA_COLOR = "#2a78d6"
MODEL_COLOR = "#eb6834"
INK = "#52514e"
GRID = "#e4e3df"

plt.rcParams.update({
    "axes.edgecolor": INK, "axes.labelcolor": INK, "xtick.color": INK, "ytick.color": INK,
    "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.6,
    "axes.spines.top": False, "axes.spines.right": False,
    "font.size": 10, "axes.titlesize": 11, "legend.frameon": False,
})

# ----- estimators (pure)

def fit_power_law(x: np.ndarray) -> dict[str, float]:
    """Continuous MLE (Clauset et al. 2009) with  $x_{\min} = \min(x)$ :  $\alpha = 1 + n / \sum \ln(x/x_{\min})$ """
    x_min = float(x.min())
    alpha = 1.0 + len(x) / float(np.log(x / x_min).sum())
    return {"alpha": alpha, "x_min": x_min}

def fit_exponential(x: np.ndarray) -> dict[str, float]:
    """MLE:  $\lambda = 1 / \text{mean}$ """
    return {"lambda": 1.0 / float(x.mean())}

def fit_uniform(x: np.ndarray) -> dict[str, float]:
    """MLE:  $[a, b] = [\min, \max]$ """
    return {"a": float(x.min()), "b": float(x.max())}

def fit_normal(x: np.ndarray) -> dict[str, float]:
```

```

"""mu = sample mean, sigma = sample standard deviation (ddof = 1)."""
return {"mu": float(x.mean()), "sigma": float(x.std(ddof=1))}

def sample_models(x: np.ndarray, rng: np.random.Generator) -> dict[str, tuple[str, np.ndarray]]:
    """Draw len(x) values from each fitted model; returns {key: (label, sample)}."""
    n = len(x)
    pl, ex, un, no = fit_power_law(x), fit_exponential(x), fit_uniform(x), fit_normal(x)
    # Inverse-CDF sampling of the Pareto form:  $x = x_{min} * (1 - u)^{-1 / (\alpha - 1)}$ .
    pl_sample = pl["x_min"] * (1.0 - rng.random(n)) ** (-1.0 / (pl["alpha"] - 1.0))
    return {
        "powerlaw": (f"power law  $\alpha$ ={pl['alpha']:.3f},  $x_{min}$ ={pl['x_min']:g}", pl_sample),
        "exponential": (f"exponential  $\lambda$ ={ex['lambda']:.4f}",
                        rng.exponential(1.0 / ex["lambda"], n)),
        "uniform": (f"uniform [a, b]={un['a']:g}, {un['b']:g}",
                    rng.uniform(un["a"], un["b"], n)),
        "normal": (f"normal  $\mu$ ={no['mu']:.3f},  $\sigma$ ={no['sigma']:.3f}",
                   rng.normal(no["mu"], no["sigma"], n)),
    }

# ----- plotting

def ccdf(x: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """P(X >= x) at each sorted value - a power law is a straight line on log-log."""
    xs = np.sort(x)
    return xs, 1.0 - np.arange(len(xs)) / len(xs)

def ecdf(x: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    xs = np.sort(x)
    return xs, np.arange(1, len(xs) + 1) / len(xs)

def qq_panel(ax: plt.Axes, data: np.ndarray, sample: np.ndarray, log: bool) -> None:
    q = np.linspace(0.005, 0.995, 199)
    dq, sq = np.quantile(data, q), np.quantile(sample, q)
    if log:
        keep = sq > 0 # normal / exponential samples can go <= 0; log axes cannot show them
        dq, sq = dq[keep], sq[keep]
        ax.set_xscale("log")
        ax.set_yscale("log")
    ax.scatter(dq, sq, s=12, color=MODEL_COLOR, zorder=3)
    lo, hi = min(dq.min(), sq.min()), max(dq.max(), sq.max())
    ax.plot([lo, hi], [lo, hi], color=INK, lw=1, ls="--", label="y = x (perfect fit)")
    ax.set(xlabel="data quantile", ylabel="model-sample quantile", title="QQ plot")
    ax.legend(loc="upper left")

def save(fig: plt.Figure, name: str) -> None:
    fig.tight_layout()
    fig.savefig(FIG_DIR / f"{name}.png", dpi=160)
    plt.close(fig)

def plot_airport_data(x: np.ndarray) -> None:
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.8))
    # Routes are integers, so plot P(X = k) per value instead of log bins (which leave gaps).
    values, counts = np.unique(x, return_counts=True)
    ax1.scatter(values, counts / len(x), s=12, color=DATA_COLOR)
    ax1.set(xscale="log", yscale="log", xlabel="routes per airport (log)",
            ylabel="P(X = k) (log)", title="PDF (probability mass per value)")

```

```

ax2.plot(*ccdf(x), color=DATA_COLOR, lw=2, label="data")
ax2.set(xscale="log", yscale="log", xlabel="routes per airport (log)",
        ylabel="P(X ≥ x) (log)", title="CCDF")
fig.suptitle(f"Airport routes: empirical distribution (n = {len(x)})")
save(fig, "p1a_0_data")

def plot_airport_model(x: np.ndarray, key: str, label: str, sample: np.ndarray) -> None:
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.8))
    ax1.plot(*ccdf(x), color=DATA_COLOR, lw=2, label="data")
    # CCDF over the FULL sample, then hide x <= 0 (log axis); filtering first would renormalise.
    xs, ps = ccdf(sample)
    ax1.plot(xs[xs > 0], ps[xs > 0], color=MODEL_COLOR, lw=2, label="model sample")
    ax1.set(xscale="log", yscale="log", xlabel="routes per airport (log)",
            ylabel="P(X ≥ x) (log)", title="CCDF: data vs model sample")
    ax1.legend(loc="lower left")
    qq-panel(ax2, x, sample, log=True)
    fig.suptitle(f"Airport routes vs {label}")
    save(fig, f"p1a_{key}")

def plot_movie_data(x: np.ndarray) -> None:
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.8))
    ax1.hist(x, bins=np.arange(1.85, 8.65, 0.1), density=True, color=DATA_COLOR)
    ax1.set(xlabel="average vote", ylabel="density",
            title="PDF, histogram (bin = 0.1, one per vote value)")
    ax2.plot(*ecdf(x), color=DATA_COLOR, lw=2)
    ax2.set(xlabel="average vote", ylabel="P(X ≤ x)", title="CDF")
    fig.suptitle(f"Movie votes: empirical distribution (n = {len(x)})")
    save(fig, "p1b_0_data")

def plot_movie_model(x: np.ndarray, key: str, label: str, sample: np.ndarray) -> None:
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.8))
    # A power-law sample reaches the thousands; clip the PDF view and say how much is hidden.
    lo = min(x.min(), np.quantile(sample, 0.001))
    hi = min(max(x.max(), np.quantile(sample, 0.999)), VOTE_VIEW_MAX)
    # Votes sit on a 0.1 grid; bin edges between grid points stop bins catching 1 or 2 values.
    bins = np.arange(np.floor(lo * 10) / 10 - 0.05, hi + 0.2, 0.2)
    width = bins[1] - bins[0]
    # Weights (not density=True) keep each curve normalised to its whole sample, even if clipped.
    ax1.hist(x, bins=bins, weights=np.full(len(x), 1 / (len(x) * width)), color=DATA_COLOR,
            alpha=0.55, label="data")
    ax1.hist(sample, bins=bins, weights=np.full(len(sample), 1 / (len(sample) * width)),
            histtype="step", color=MODEL_COLOR, lw=2, label="model sample")
    hidden = float(np.mean((sample < bins[0]) | (sample > bins[-1])))
    if hidden > 0.01:
        ax1.text(0.98, 0.58, f"{hidden:.0%} of the sample\nlies beyond this axis",
                transform=ax1.transAxes, ha="right", color=INK)
    ax1.set(xlabel="average vote", ylabel="density", title="PDF: data vs model sample")
    ax1.legend(loc="upper right")
    qq-panel(ax2, x, sample, log=hi == VOTE_VIEW_MAX)
    fig.suptitle(f"Movie votes vs {label}")
    save(fig, f"p1b_{key}")

# ----- main

def report(name: str, x: np.ndarray) -> None:
    print(f"\n== {name} (n = {len(x)}, mean = {x.mean():.4f}, median = {np.median(x):.4f})")
    for fit in (fit_power_law, fit_exponential, fit_uniform, fit_normal):
        print(f" {fit.__name__:16s} {fit(x)}")

```

```

def main() -> None:
    FIG_DIR.mkdir(exist_ok=True)
    rng = np.random.default_rng(SEED)
    airports = pd.read_csv(DATA_DIR / "01_airport_routes.csv")["NumberOfRoutes"].to_numpy(float)
    movies = pd.read_csv(DATA_DIR / "01_movie_votes.csv")["AverageVote"].to_numpy(float)

    report("airport routes", airports)
    plot_airport_data(airports)
    for i, (key, (label, sample)) in enumerate(sample_models(airports, rng).items(), start=1):
        print(f" KS vs {key:12s} D = {ks_2samp(airports, sample).statistic:.3f}")
        plot_airport_model(airports, f"{i}_{key}", label, sample)

    report("movie votes", movies)
    plot_movie_data(movies)
    for i, (key, (label, sample)) in enumerate(sample_models(movies, rng).items(), start=1):
        print(f" KS vs {key:12s} D = {ks_2samp(movies, sample).statistic:.3f}")
        plot_movie_model(movies, f"{i}_{key}", label, sample)

if __name__ == "__main__":
    main()

```

Problem 2: hw2/code/p2.py

```

"""
Problem 2: age, first digit and last digit of IMDB-WIKI birth dates.

Parsing decisions (checked against the raw year counts, not assumed):
- Dates are "dd-Mon-yy" or "dd-Mon-yyyy". Every 2-digit year is read as 19yy: IMDB-WIKI was
  crawled in 2015, so "20"- "24" cannot be 2020-2024, and the counts fall smoothly from the
  1920s down to "00" with no break that would mark a century switch.
- Year "0000" is a missing-value placeholder and is dropped.
- Age = completed years on REFERENCE_DATE (the 2015 crawl), kept only if 1 <= age <= 99.

Run from the repo root: uv run python hw2/code/p2.py
"""

from pathlib import Path

import matplotlib

matplotlib.use("Agg") # headless: write PNGs only, no GUI backend needed
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

HW_DIR = Path(__file__).resolve().parent.parent
DATA_PATH = HW_DIR / "00_doc" / "01_DoB.txt"
FIG_DIR = HW_DIR / "figures"
REFERENCE_DATE = pd.Timestamp("2015-01-01")

DATA_COLOR = "#2a78d6"
BENFORD_COLOR = "#eb6834"
INK = "#52514e"
GRID = "#e4e3df"

plt.rcParams.update({
    "axes.edgecolor": INK, "axes.labelcolor": INK, "xtick.color": INK, "ytick.color": INK,
    "axes.grid": True, "axes.axisbelow": True, "grid.color": GRID, "grid.linewidth": 0.6,
    "axes.spines.top": False, "axes.spines.right": False,
    "font.size": 10, "axes.titlesize": 11, "legend.frameon": False,

```

```

}))

def parse_birth_dates(raw: pd.Series) -> pd.Series:
    """dd-Mon-yy / dd-Mon-yyyy -> Timestamp; 2-digit years -> 19yy; year 0000 -> NaT."""
    parts = raw.str.strip().str.split("-", expand=True)
    year = parts[2].astype(int)
    year = year.where(parts[2].str.len() == 4, 1900 + year)
    year = year.where(year > 0)
    text = parts[0] + "-" + parts[1] + "-" + year.astype("Int64").astype(str)
    return pd.to_datetime(text, format="%d-%b-%Y", errors="coerce")

def compute_ages(birth: pd.Series, on: pd.Timestamp) -> pd.Series:
    """Completed years on 'on', restricted to 1-99."""
    had_birthday = (birth.dt.month < on.month) | (
        (birth.dt.month == on.month) & (birth.dt.day <= on.day))
    age = on.year - birth.dt.year - (~had_birthday).astype(int)
    age = age.dropna().astype(int)
    return age[(age >= 1) & (age <= 99)]

def share_per_value(values: pd.Series, support: range) -> pd.Series:
    return values.value_counts(normalize=True).reindex(support, fill_value=0.0)

def plot_bars(share: pd.Series, uniform: float | None, title: str, xlabel: str, name: str,
              benford: pd.Series | None = None) -> None:
    fig, ax = plt.subplots(figsize=(8, 3.6))
    ax.bar(share.index, share.to_numpy(), width=0.8, color=DATA_COLOR, edgecolor="white", lw=0.5,
          label="data")
    if uniform is not None:
        ax.axhline(uniform, color=INK, ls="--", lw=1.2, label=f"uniform = {uniform:.3f}")
    if benford is not None:
        ax.plot(benford.index, benford.to_numpy(), color=BENFORD_COLOR, marker="o", ms=5, lw=2,
              label="Benford log10(1 + 1/d)")
    if uniform is not None or benford is not None:
        ax.set_ylim(top=max(share.max(), 0.0 if benford is None else benford.max()) * 1.3)
        ax.legend(loc="upper right", ncol=3) # headroom + one row keeps the legend off the bars
    ax.set(title=title, xlabel=xlabel, ylabel="share of actors")
    fig.tight_layout()
    fig.savefig(FIG_DIR / f"{name}.png", dpi=160)
    plt.close(fig)

def main() -> None:
    FIG_DIR.mkdir(exist_ok=True)
    raw = pd.read_csv(DATA_PATH, header=None, names=["dob"], dtype=str)["dob"]
    birth = parse_birth_dates(raw)
    print(f"rows = {len(raw)}, unparseable/placeholder = {birth.isna().sum()}")
    print(f"birth years: min = {birth.dt.year.min():.0f}, max = {birth.dt.year.max():.0f}")

    age = compute_ages(birth, REFERENCE_DATE)
    first = age.astype(str).str[0].astype(int)
    last = age % 10
    print(f"ages kept (1-99) = {len(age)}, mean = {age.mean():.2f}, median = {age.median():.0f}, "
          f"min = {age.min()}, max = {age.max()}")

    first_share = share_per_value(first, range(1, 10))
    last_share = share_per_value(last, range(10))
    print("first digit share:", first_share.round(4).to_dict())
    print("last digit share: ", last_share.round(4).to_dict())

```

```

benford = pd.Series({d: float(np.log10(1 + 1 / d)) for d in range(1, 10)})
print("Benford reference:", benford.round(4).to_dict())
print(f"max |last share - 0.1| = {(last_share - 0.1).abs().max():.4f}")

plot_bars(share_per_value(age, range(1, 100)), None,
          f"Age on {REFERENCE_DATE.date()} (n = {len(age):,})", "age (years)", "p2_1_age")
plot_bars(first_share, 1 / 9, "First digit of age", "first digit", "p2_2_first_digit",
          benford=benford)
plot_bars(last_share, 1 / 10, "Last digit of age", "last digit", "p2_3_last_digit")

if __name__ == "__main__":
    main()

```